

Reproducing DoRA: Weight-Decomposed Low-Rank Adaptation

Keng-Lien Lin, Ellen Foreman, Yiyi Wu

https://github.com/KengLL/DoRA_PEFT

1 Introduction

Fine-tuning is essential for adapting models to downstream tasks. Traditional full fine-tuning requires updating all parameters, which becomes expensive as model sizes grow, motivating the development of parameter-efficient fine-tuning (PEFT) methods like Low-Rank Adaptation (LoRA), which take a small subset of parameters for adapting to downstream tasks [2]. While LoRA injects trainable low-rank matrices, it falls short of full fine-tuning performance due to its coupled treatment of weight magnitude and direction.

DoRA: Weight-Decomposed Low-Rank Adaptation by Liu et al. addresses this [1] by decomposing pretrained weights into magnitude and direction components, tuning each independently. This decomposition allows DoRA to more closely mirror the learning behavior of full fine-tuning compared to LoRA with negligible additional overhead. This project reproduces the results of DoRA from Liu et al.

2 Chosen Result

We choose to reproduce the accuracy of the LoRA and DoRA on 8 commonsense reasoning tasks (BoolQ, PIQA, SIQA, HellaSwag, WinoGrande, ARC-e, ARC-c, OBQA), specifically the LLaMA3-8B rows in Table 1 of the original paper [1], as it is the closest available architecture to our LLaMA3.2-1B model. These results are significant as they show DoRA surpasses ChatGPT and outperforms LoRA by $\sim 4.4\%$ on average [1].

Beyond accuracy, this result directly validates the paper’s core claim that decoupling magnitude and direction allows the model to more closely mirror full fine-tuning, especially under constrained parameter budgets where LoRA’s coupling becomes most restrictive. We also reproduce the LoRA and DoRA magnitude and direction updates in Figure 2 of the original paper [1] to visualize this decoupling effect.

Model	PEFT Method	# Params (%)	BoolQ	PIQA	SIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA	Avg.
ChatGPT	-	-	73.1	85.4	68.5	78.5	66.1	89.8	79.9	74.8	77.0
LLaMA3-8B	LoRA	0.70	70.8	85.2	79.9	91.7	84.3	84.2	71.2	79.0	80.8
	DoRA [†] (Ours)	0.35	74.5	88.8	80.3	95.5	84.7	90.1	79.1	87.2	85.0
	DoRA (Ours)	0.71	74.6	89.3	79.9	95.5	85.6	90.5	80.4	85.8	85.2

Table 1: Accuracy of DoRA, LoRA on 8 commonsense reasoning tasks in the original paper [1].

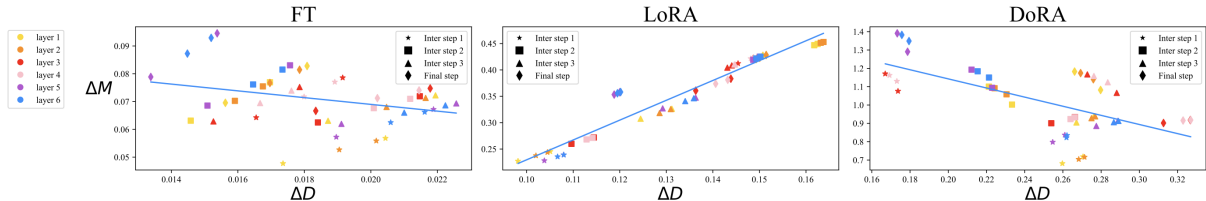


Figure 1: Magnitude and direction updates in of FT, LoRA and DoRA in Figure 2 of the original paper [1].

3 Methodology

3.1 Configuration

Our experiment uses the LLaMA-3.2-1B model [3] instead of the LLaMA-8B in the paper and takes a 20K training slice from the Commonsense170K [4] dataset due to compute constraints. We evaluate on ranks 8, 4, and 2, instead of 16 in the original paper to reveal more rank-sensitive accuracy differences between LoRA and DoRA, as our smaller model saturates at higher ranks. Remaining hyperparameters follow the original paper: $\alpha = 2 \times \text{rank}$, dropout = 0.05, AdamW optimizer with $\text{lr} = 2\text{e-}4$, 3 epochs, batch size 16, and gradient

accumulation of 4 [1]. We apply DoRA to the query (q), key (k), value (v), and feed-forward projection (up, down) matrices of each transformer layer. We used 3 seeded runs to increase statistical stability.

3.2 Model Architecture

DoRA computes the adapted weight as $W' = m \cdot \frac{W_0 + sBA}{\|W_0 + sBA\|_{\text{row}}}$, where $m \in \mathbb{R}^{d \times 1}$ is a learnable magnitude vector, BA is LoRA’s low-rank direction update, and $s = \alpha/r$ is the LoRA scaling factor.

Our implementation does this computation through three parallel paths, as shown in Figure 2.

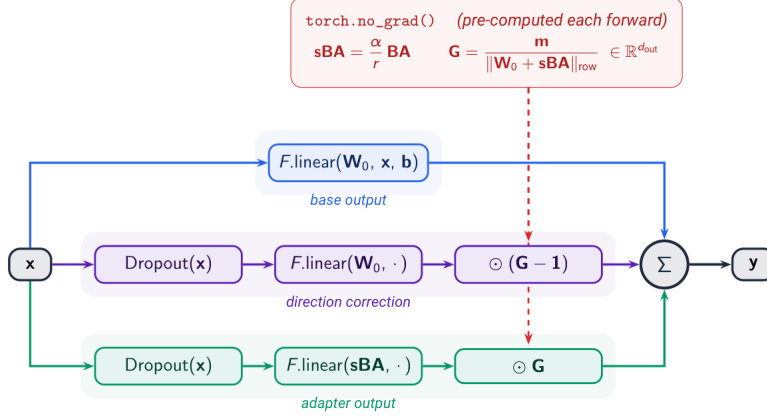


Figure 2: Forward pass diagram of our DoRA implementation.

In Figure 2, the input x passes through dropout, then splits into three parallel paths. The base path computes $F.\text{linear}(W_0, x, b)$ using the frozen pretrained weights. This result is reused, avoiding materializing the full adapted weight matrix W' . The direction correction path computes $F.\text{linear}(W_0, \cdot)$ scaled by $(G - 1)$, and the adapter output path computes $F.\text{linear}(sBA, \cdot)$ scaled by G , where $G = \frac{m}{\|W_0 + sBA\|_{\text{row}}}$ is the per-row magnitude rescaling factor.

Crucially, G and sBA are computed inside a `torch.no_grad()` block, meaning the row-norm denominator $\|W_0 + sBA\|_{\text{row}}$ is detached from the computational graph and receives no gradient during backpropagation. Following §4 of the original paper [1], this is a deliberate design choice motivated primarily by memory efficiency: detaching the norm denominator drastically reduces gradient graph memory consumption, with negligible impact on accuracy. The three paths are then summed to produce the final output y , which is equivalent to $W'x$ without ever explicitly constructing W' .

4 Results & Analysis

PEFT Method	Rank	# Params(%)	BoolQ	Hella.	Wino.	ARC-e	ARC-c	OBQA	PIQA	SIQA	Avg.
Base	—	0.000%	37.8	44.6	51.6	56.6	35.8	31.6	78.2	16.2	44.1
LoRA	8	0.309%	37.4 ± 0.4	46.8 ± 1.1	50.2 ± 0.5	63.7 ± 0.6	39.5 ± 3.0	34.2 ± 0.5	74.7 ± 0.5	16.5 ± 0.5	45.4
DoRA (ours)		0.326%	37.8 ± 0.4	46.8 ± 0.8	52.4 ± 0.9	64.4 ± 2.0	37.4 ± 0.8	33.3 ± 0.3	75.1 ± 0.9	16.1 ± 0.3	45.4
DoRA (lib)		0.326%	38.7	46.3	50.6	63.3	39.8	35	75.3	16.3	45.7
LoRA	4	0.155%	37.6 ± 0.7	46.1 ± 1.2	50.0 ± 1.2	63.3 ± 2.1	37.2 ± 2.0	33.5 ± 1.1	75.2 ± 0.3	16.8 ± 0.6	45.0
DoRA (ours)		0.172%	38.5 ± 0.4	46.4 ± 0.2	50.7 ± 0.7	63.9 ± 1.7	35.9 ± 1.1	33.7 ± 0.4	75.1 ± 0.4	16.8 ± 0.7	45.1
LoRA	2	0.077%	38.4 ± 1.2	45.9 ± 0.7	50.1 ± 1.0	62.2 ± 1.9	34.8 ± 2.3	32.4 ± 0.7	74.8 ± 0.3	16.7 ± 0.2	44.4
DoRA (ours)		0.095%	38.6 ± 0.8	46.4 ± 0.4	50.2 ± 0.3	63.8 ± 1.1	35.3 ± 0.8	33.5 ± 0.6	75.0 ± 0.2	16.8 ± 0.6	45.0

Table 2: Accuracy (%) of LoRA and DoRA (ours) on 8 commonsense reasoning benchmarks across ranks 2, 4, and 8, with mean ± std across 3 seeded runs.

As shown in Table 2, our DoRA implementation exceeds LoRA across ranks 8, 4, and 2, with the advantage becoming especially pronounced at lower ranks where LoRA degrades more sharply, which follows the original paper’s trends [1]. We verified correctness of our implementation against the DoRA library at rank 8. We initially tested rank 16 to match the original paper, but the performance gap was minimal, likely due to capacity saturation of our 1B model, leaving little room for DoRA’s decomposition to provide benefit. The variance creates overlaps sometimes between DoRA and LoRA, making differences not statistically significant. The original paper does not report variance or run count, so it is unclear if similar overlaps exist.

To examine decoupling, we plot the magnitude and direction updates for LoRA and DoRA in Figure 3.

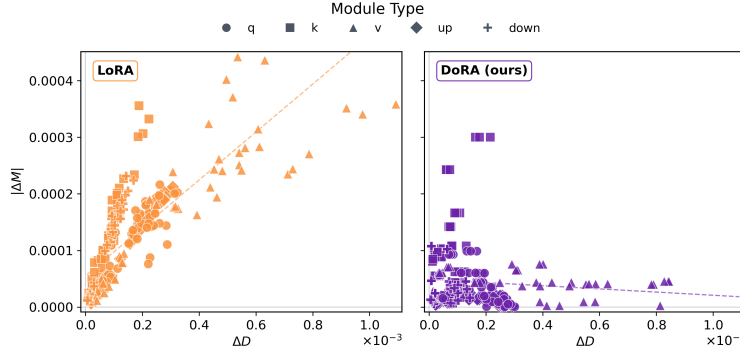


Figure 3: LoRA and DoRA magnitude and direction updates at rank 8.

LoRA’s trend line shows a strong positive correlation between magnitude and direction update size, indicating the components are entangled. Meanwhile, DoRA exhibits a nearly flat and slightly negative trend line, showing magnitude changes remain small regardless of direction update size, confirming successful decoupling and enabling more targeted adaptation, consistent with the original paper [1].

Figure 4 shows the average accuracy and variance across ranks for LoRA and DoRA.

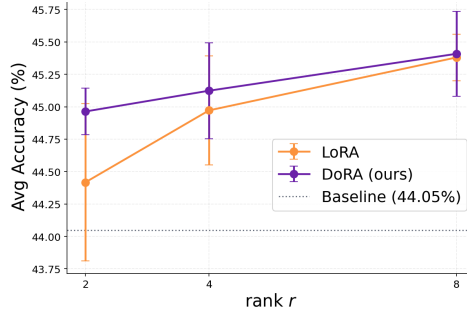


Figure 4: Average accuracy and variance of LoRA, DoRA, and baseline across rank 2, 4, and 8.

Notably, LoRA drops below the unadapted baseline (44.05%) at rank 2, while DoRA remains above it, demonstrating that DoRA’s magnitude-direction decomposition provides meaningful stability under severe parameter constraints, matching the original paper’s findings [1].

5 Reflections

Our DoRA implementation consistently outperforms LoRA across all ranks, making it a practical drop-in replacement with no architectural change and adding marginal extra parameters. Our results are consistent with the original paper’s findings [1] despite our model size difference. We learned that higher ranks for small models could experience capacity saturation, reducing DoRA’s benefits. Fine-tuning is sensitive to initialization, especially at lower ranks. Our three seeded runs provide statistical stability, but may still be

insufficient to characterize the variance.

For future directions, we hope to continue investigating other PEFT methods such as magnitude-aware PEFT methods (MAP). We also hope to extend our investigation to domains beyond text, including vision and audio, to assess whether DoRA’s advantages persist across domains.

References

- [1] Liu, S.-Y. et al. (2024). DoRA: Weight-Decomposed Low-Rank Adaptation. *arXiv:2402.09353*.
- [2] Hu, E. et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
- [3] Meta AI (2024). Llama 3.2. <https://huggingface.co/meta-llama/Llama-3.2-1B>.
- [4] He, Z. (2024). Commonsense170K. https://huggingface.co/datasets/zwhe99/commonsense_170k.